

Experiment 6

Student Name: Yash Kotra

Branch: BE CSE

Semester: 6th

Subject Name: Full Stack Development

UID: 23BCS10599

Section/Group: KRG 3A

Date of Performance: 02/03/26

Subject Code: 23CSH-309

Aim:

To design and implement a Spring Boot REST API using Layered Architecture, applying Inversion of Control (IoC) and Dependency Injection (DI) to build a structured backend for the HealthHub application, including CRUD operations, DTO mapping, validation, and global exception handling.

Objective:

After completing this experiment, students will be able to:

1. Understand Spring Boot fundamentals
 - Learn the role of Spring Boot in backend development.
 - Configure and run a basic Spring Boot application.
2. Apply IoC and Dependency Injection
 - Understand how Spring manages object creation and dependencies.
 - Use annotations like `@Autowired`, `@Service`, and `@Repository`.
3. Implement Layered Architecture
 - Structure the application using Controller, Service, and Repository layers.
 - Understand separation of concerns in backend design.
4. Develop RESTful APIs
 - Create CRUD operations using Spring Boot REST controllers.
 - Design APIs using proper HTTP methods (GET, POST, PUT, DELETE).

5. Use DTOs for API communication
 - Implement DTO mapping to separate internal models from API responses.
6. Implement Validation
 - Apply validation using Jakarta Validation annotations (@NotNull, @Email, etc.).
7. Handle Exceptions Globally
 - Implement global exception handling using @ControllerAdvice.
8. Build the HealthHub Backend Module
 - Develop backend services for patient/health record management.

Implementation/Code:

PatientController.java:

```
package com.example.healthhub.controller;

import com.example.healthhub.dto.PatientDTO;
import com.example.healthhub.entity.Patient;
import com.example.healthhub.service.PatientService;
import jakarta.validation.Valid;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.web.bind.annotation.*;

import java.util.List;

@RestController
@RequestMapping("/patients")
public class PatientController {

    @Autowired
    private PatientService service;

    @PostMapping
    public Patient create(@Valid @RequestBody PatientDTO dto){
```

```
Patient patient = new Patient();
patient.setName(dto.getName());
patient.setEmail(dto.getEmail());
patient.setAge(dto.getAge());
```

```
return service.save(patient);
}
```

```
@GetMapping
public List<Patient> getAll(){
    return service.getAll();
}
```

```
@GetMapping("/{id}")
public Patient getById(@PathVariable Long id){
    return service.getById(id);
}
```

```
@DeleteMapping("/{id}")
public void delete(@PathVariable Long id){
    service.delete(id);
}
}
```

PatientRepository.java:

```
package com.example.healthhub.repository;
```

```
import com.example.healthhub.entity.Patient;
import org.springframework.data.jpa.repository.JpaRepository;
```

```
public interface PatientRepository extends JpaRepository<Patient, Long> {
}
```

PatientService.java:

```
package com.example.healthhub.service;
```

```
import com.example.healthhub.entity.Patient;
import com.example.healthhub.repository.PatientRepository;
```

```
import org.springframework.beans.factory.annotation.Autowired;  
import org.springframework.stereotype.Service;
```

```
import java.util.List;
```

```
@Service
```

```
public class PatientService {
```

```
    @Autowired
```

```
    private PatientRepository repository;
```

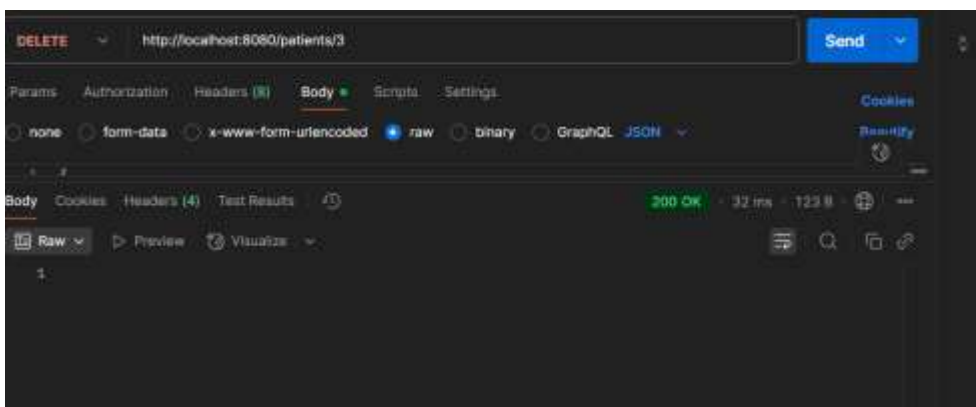
```
    public Patient save(Patient patient){  
        return repository.save(patient);  
    }
```

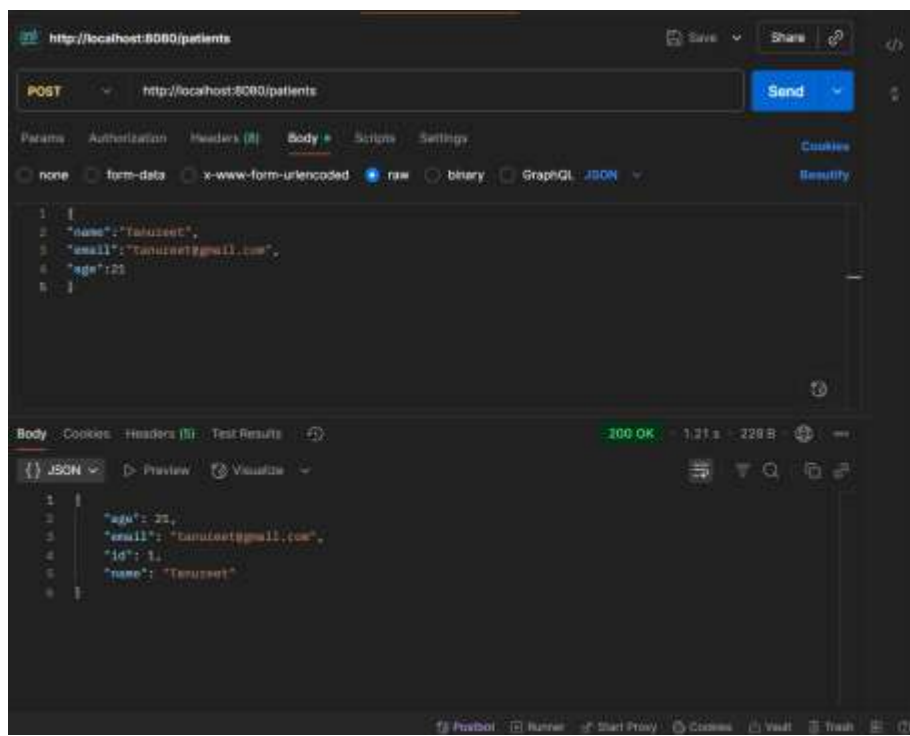
```
    public List<Patient> getAll(){  
        return repository.findAll();  
    }
```

```
    public Patient getById(Long id){  
        return repository.findById(id).orElse(null);  
    }
```

```
    public void delete(Long id){  
        repository.deleteById(id);  
    }  
}
```

Output:

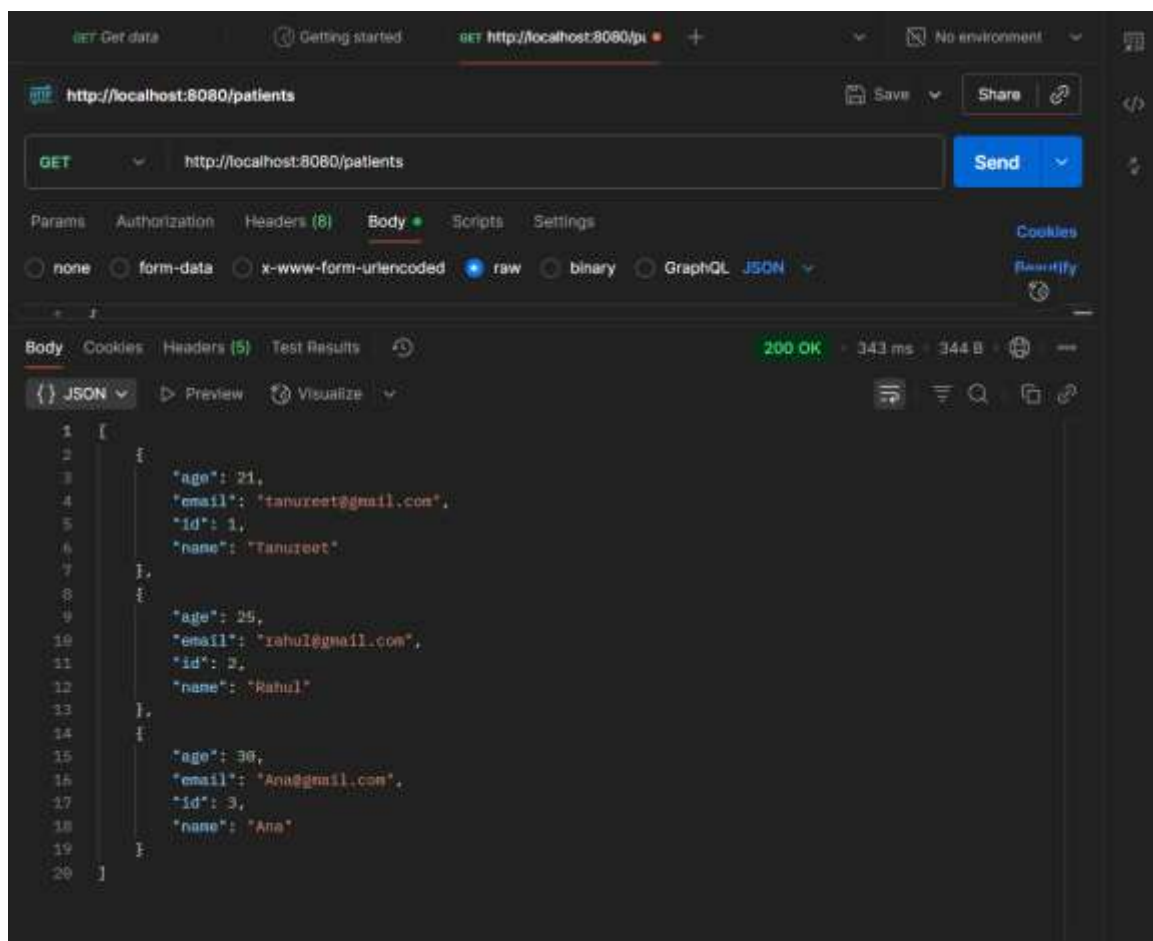




The screenshot shows the Postman interface for a POST request to `http://localhost:8080/patients`. The request body is a JSON object: `{ "name": "tanuseet", "email": "tanuseet@gmail.com", "age": 21 }`. The response is a 200 OK status with a 1.21 s response time and 228 B of data. The response body is a JSON object: `{ "age": 21, "email": "tanuseet@gmail.com", "id": 1, "name": "tanuseet" }`.

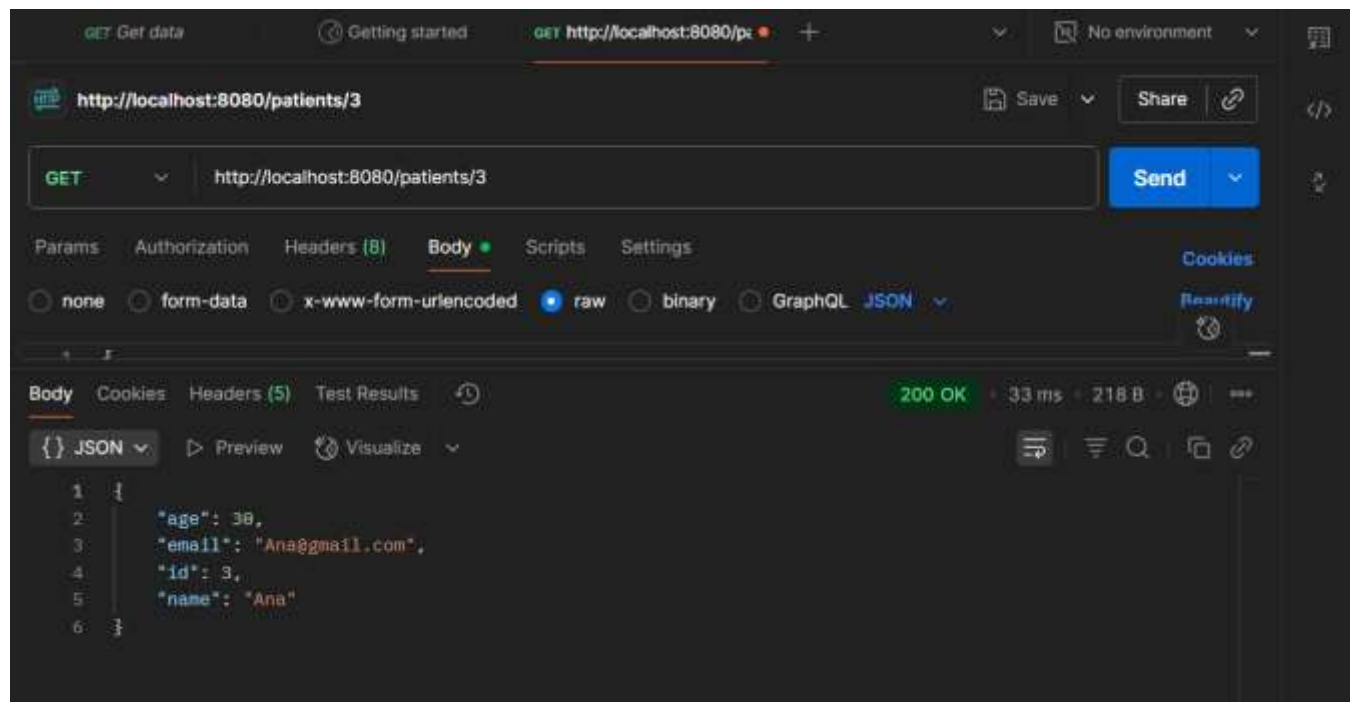
```
1 {
2   "name": "tanuseet",
3   "email": "tanuseet@gmail.com",
4   "age": 21
5 }
```

```
1 {
2   "age": 21,
3   "email": "tanuseet@gmail.com",
4   "id": 1,
5   "name": "tanuseet"
6 }
```



The screenshot shows the Postman interface for a GET request to `http://localhost:8080/patients`. The response is a 200 OK status with a 343 ms response time and 344 B of data. The response body is a JSON array of three objects: `[ { "age": 21, "email": "tanuseet@gmail.com", "id": 1, "name": "tanuseet" }, { "age": 25, "email": "rahul@gmail.com", "id": 2, "name": "rahul" }, { "age": 30, "email": "Ana@gmail.com", "id": 3, "name": "Ana" } ]`.

```
1 [
2   {
3     "age": 21,
4     "email": "tanuseet@gmail.com",
5     "id": 1,
6     "name": "tanuseet"
7   },
8   {
9     "age": 25,
10    "email": "rahul@gmail.com",
11    "id": 2,
12    "name": "rahul"
13  },
14  {
15    "age": 30,
16    "email": "Ana@gmail.com",
17    "id": 3,
18    "name": "Ana"
19  }
20 ]
```



GET http://localhost:8080/patients/3

Params Authorization Headers (8) Body Scripts Settings

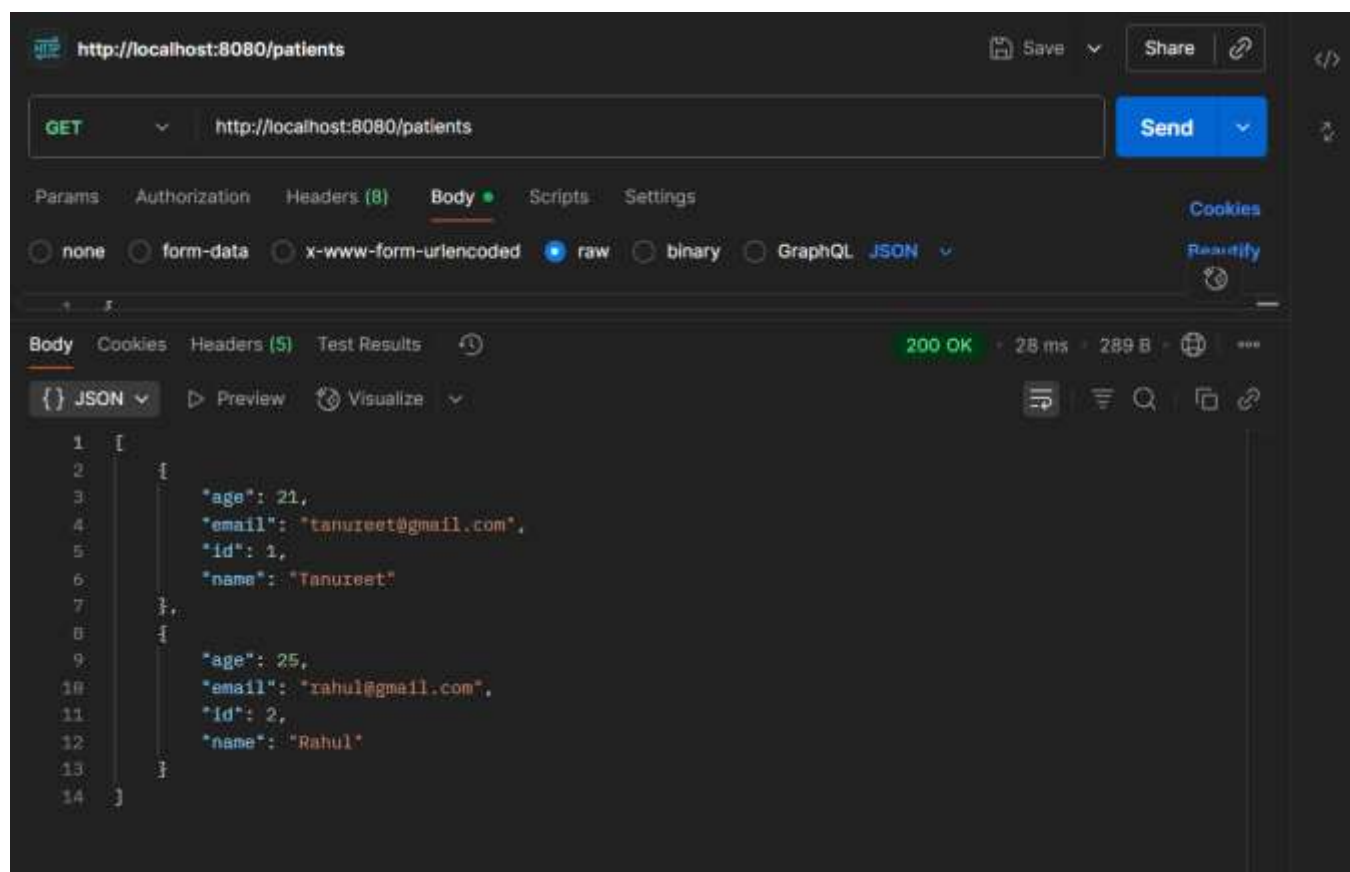
none form-data x-www-form-urlencoded raw binary GraphQL JSON

Body Cookies Headers (5) Test Results

200 OK · 33 ms · 218 B

JSON Preview Visualize

```
1 {
2   "age": 30,
3   "email": "Ana@gmail.com",
4   "id": 3,
5   "name": "Ana"
6 }
```



GET http://localhost:8080/patients

Params Authorization Headers (8) Body Scripts Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

Body Cookies Headers (5) Test Results

200 OK · 28 ms · 289 B

JSON Preview Visualize

```
1 [
2   {
3     "age": 21,
4     "email": "tanureet@gmail.com",
5     "id": 1,
6     "name": "Tanureet"
7   },
8   {
9     "age": 25,
10    "email": "rahul@gmail.com",
11    "id": 2,
12    "name": "Rahul"
13  }
14 ]
```

Learning Outcome:

- **Develop a Spring Boot application**
Create and run a basic Spring Boot project and understand its role in modern backend development.
- **Apply Inversion of Control (IoC) and Dependency Injection (DI)**
Implement IoC and DI concepts using Spring annotations such as `@Autowired`, `@Service`, and `@Repository` to manage dependencies efficiently.
- **Design applications using Layered Architecture**
Structure backend applications into Controller, Service, and Repository layers, ensuring proper separation of concerns and maintainable code.
- **Create RESTful APIs with CRUD functionality**
Develop REST APIs using appropriate HTTP methods (GET, POST, PUT, DELETE) to perform Create, Read, Update, and Delete operations.
- **Implement DTO-based data transfer**
Use Data Transfer Objects (DTOs) to separate internal entity models from API request and response structures.